Research Square

Preprints are preliminary reports that have not undergone peer review. They should not be considered conclusive, used to inform clinical practice, or referenced by the media as validated information.

Shallow and Robust QAOA: Improving Feasibility and Hardware Performance via Linear-Chain and Ramp Schedules

Talha Azfar

Rensselaer Polytechnic Institute

Ruimin Ke

ker@rpi.edu

Rensselaer Polytechnic Institute

Article

Keywords: Quantum Approximate Optimization Algorithm, Vehicle Routing Problem, Combinatorial optimization

Posted Date: January 5th, 2026

DOI: https://doi.org/10.21203/rs.3.rs-8297477/v1

License: This work is licensed under a Creative Commons Attribution 4.0 International License. Read Full License

Additional Declarations: No competing interests reported.

Shallow and Robust QAOA: Improving Feasibility and Hardware Performance via Linear-Chain and Ramp Schedules

Talha Azfar, Ruimin Ke*
Rensselaer Polytechnic Institute, Troy, NY, USA

Abstract

The Quantum Approximate Optimization Algorithm (QAOA) is known to struggle on near-term hardware due to constraint handling, parameter search overhead, and tight depth limits. We address this with a hardware-aware co-design: a linear-ramp parameter schedule paired with a Linear-Chain (LC) ansatz that restricts ZZ interactions to nearest neighbors, lowering swaps and depth. We benchmark Linear-Ramp LC-QAOA against standard QAOA, Two-Step QAOA, and Grover-Mixer QAOA on small VRP instances across ideal simulation, Aer simulation, and IBM Eagle/Heron runs, measuring feasibility, solution quality, and hardware efficiency. Linear-ramp improves convergence and feasibility; LC reduces two-qubit gates and boosts noise robustness. Grover-Mixer and Two-Step can aid constraints in theory but are hampered in practice by deeper circuits. On hardware, overall feasibility remains $< 1\%$, yet LC-QAOA with $XpXm$ dynamical decoupling more than doubles feasibility and recovers the optimum in several trials. We also show efficient scaling, including a larger case with ramp-initialized LC-QAOA plus CVaR. The results underscore hardware-aligned ansätze and structured schedules as a promising path.

Keywords: Quantum Approximate Optimization Algorithm; Vehicle Routing Problem; Combinatorial optimization

Quantum approximate optimization algorithms (QAOA) [1] offer a compelling route to near-term combinatorial optimization, yet their practical utility hinges on three intertwined factors: constraint handling, classical-parameter overhead, and hardware-realistic circuit depth on noisy, connectivity-limited devices. Despite many

*Corresponding Author, email: ker@rpi.edu

creative extensions – constraint-preserving mixers, Grover-style state preparation, warm starts, and schedule heuristics – what is missing is a systematic, end-to-end comparison that spans ideal simulation, shot-based sampling, and execution on real superconducting hardware, while reporting feasibility, optimality, and transpiled two-qubit depth under the same problem family and evaluation protocol.

Contemporary QAOA designs may be organized into three complementary categories that address concerns of feasibility, circuit depth, and robust initialization: *(1)* Constraint-preserving approaches (e.g., XY mixers, Grover-mixer with Dicke-state seeding) that aim to keep the evolution within (or near) the feasible manifold [2, 3]; *(2)* Schedule/initialization strategies (e.g., linear-ramp schedules and parameter transfer) that reduce classical search effort and stabilize training [4]; and *(3)* Hardware-aware ansätze that trade expressiveness for shallow, topology-friendly circuits (e.g., Linear-Chain QAOA that restricts ZZ terms to neighbors to cut swap overhead) [5]. These threads jointly shape feasibility and fidelity on current devices.

Numerous other promising QAOA approaches have been proposed, for example, multi-level QAOA [6] is a coarse-graining strategy for scaling to much larger problems, but it requires a hierarchical problem-specific encoding which adds many hyperparameters, and does not directly reduce two-qubit depth. ADAPT-QAOA [7] builds an adaptive ansätz that iteratively adds problem-informed operators, but greatly increases classical evaluations while including trial-dependent circuit structures. This makes feasibility and depth comparisons difficult, and instead shift the focus towards operator-pool design. Noise-Directed Adaptive Remapping [8] is primarily a device calibration and mapping tactic rather than an algorithmic change to mixers, schedules, or ansätze. It is highly dependent on hardware characteristics, which reduces portability and confounds cross-hardware comparisons, orthogonal to our goal of assessing generic strategies that lower depth and maintain feasibility while easing optimization.

We therefore investigate five representatives spanning the three complementary categories on the Vehicle Routing Problem (VRP): Standard QAOA, Two-Step QAOA, Grover-Mixer QAOA, Linear-Ramp-Initialized QAOA (LRI-QAOA), and a Linear-Chain (LC) ansätz. The VRP provides a demanding yet structured testbed: it couples dense combinatorial structure (all-to-all connection) with hard feasibility constraints (degree, flow, subtours) [9] and a cost landscape known to challenge shallow variational circuits [10]. The problem is also a central logistics primitive with high practical salience, making feasibility rates and circuit depth trends directly informative for real deployments. While QAOA benchmarks often default to Max-Cut, it underrepresents real operational constraints and topology-induced depth; we therefore study VRP, where constraint coupling is central and prior quantum results are largely limited beyond toy instances.

Feasibility-preserving encodings for the VRP that rely on ancilla and multi-controlled gates remain hardware intensive [11]. In contrast, penalty-based encodings fold constraints into the cost Hamiltonian as QUBOs or Ising models [12, 13], but often yield

rugged landscapes and larger effective instances. Beyond gate-model approaches, quantum annealing and hybrid methods decompose VRP into QUBO subproblems or use constrained models (CQM), with classical pre/post-processing to enforce time windows or repair routes [14, 15, 16].

Our study explicitly bridges algorithmic and hardware considerations. Linear-ramp schedules and parameter transfer address classical optimization overhead by providing robust, instance-transferable angles, while hardware-aligned LC ansätze lower two-qubit depth on heavy-hex topologies, improving noise robustness even at modest depths. Within this co-design view, we also test lightweight error suppression (dynamical decoupling) and contrast it with heavier post-processing mitigation to clarify what actually helps in the QAOA loop on today's processors.

We assess each method's viability for the VRP from idealized simulation to execution on IBM Eagle (Rensselaer) and Heron (Fez) processors. In addition, we transfer the learned parameters from simulation to hardware, and show that, in combination with noise suppression techniques, LRI-QAOA is capable of providing high quality solution on current quantum computers. To address the scaling problem we study the efficacy of high p LC-QAOA, its two-qubit depth versus problem size, and successfully combine it with linear-ramp initialization on a larger problem. Finally, we include the Conditional Value at Risk (CVaR) objective [17] to boost the optimization. An overview of the workflow is shown in Figure 1. The central theme emerging from our study is that NISQ performance is governed less by formal expressibility and more by circuit depth, connectivity, and noise resilience. In particular, linear-ramp schedules provide robust parameterization and LC-QAOA substantially lowers two-qubit depth, enabling reliable hardware runs at scales where other ansätze fail to compile or decohere.

1 Results

We evaluate the five QAOA variants along four key dimensions: *(1)* their ability to produce feasible solutions that satisfy all VRP constraints, *(2)* the quality of the solutions (finding the known optimum route), *(3)* the quantum circuit depth and gate requirements, and *(4)* robustness to noise, including performance on real hardware with and without error mitigation. In the figures and tables below, we use the following abbreviations: QAOA (Standard QAOA), Two-Step (Two-Step QAOA), GM-QAOA (Grover-Mixer QAOA), LRI-QAOA (Linear-Ramp Initialized QAOA), and LC-QAOA (Linear-Chain QAOA).

All experiments were implemented using the Qiskit SDK (version 2.1.1) [18] within the Qiskit Runtime environment and executed on a combination of classical simulators and real quantum processors. Problem formulations derived from the classical DOcplex model were first converted into a QuadraticProgram, which was then mapped to an Ising Hamiltonian using Qiskit optimization's QUBO conversion and `to_ising()` methods. The resulting Hamiltonians were represented internally as SparsePauliOp

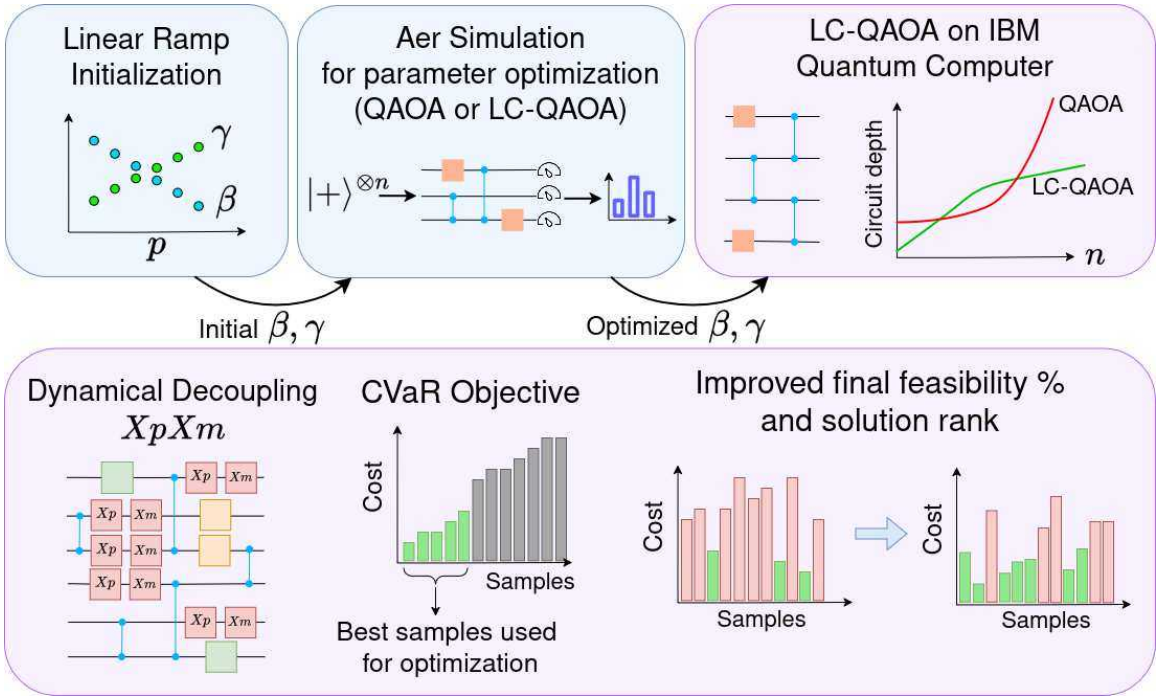


Figure 1: Algorithm-hardware co-design: linear ramp initialization and sim-to-hardware parameter transfer curb classical search overhead, while hardware-aligned linear-chain ansätze reduce two-qubit depth on heavy-hex topologies. The bottom part shows lightweight error suppression in hardware with dynamical decoupling, and improved optimization with CVaR resulting in higher quality solutions.

objects for efficient manipulation, filtering, and circuit construction. These operators served as inputs to the QAOAAnsatz and EstimatorV2 classes for expectation value evaluation and parameter optimization, followed by the SamplerV2 for measurement and solution sampling. We used COBYQA (a derivative-free optimizer) to tune the QAOA parameters for each method, targeting a low expectation value of the VRP cost Hamiltonian. For each set of optimized parameters, we sampled 10,000 measurement outcomes (shots) to estimate solution statistics.

To assess the performance of different QAOA variants, we consider a representative Vehicle Routing Problem (VRP) instance comprising a single depot, three customer nodes, and two vehicles as depicted in Figure 2. The instance is kept deliberately small to remain within the limits of local simulation and near-term quantum hardware while retaining the essential combinatorial structure of the problem. For the validation of the correct solution rank we solve the problem using the CPLEX library in Python beforehand, which takes only a few seconds at this scale.

Customer locations are randomly distributed on a two-dimensional Euclidean plane, and pairwise edge weights are computed as squared Euclidean distances between customer locations. This choice preserves the relative ordering of routes while

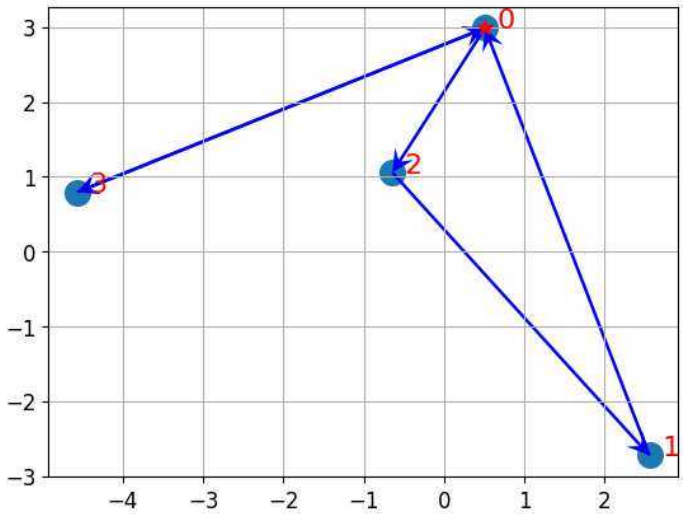


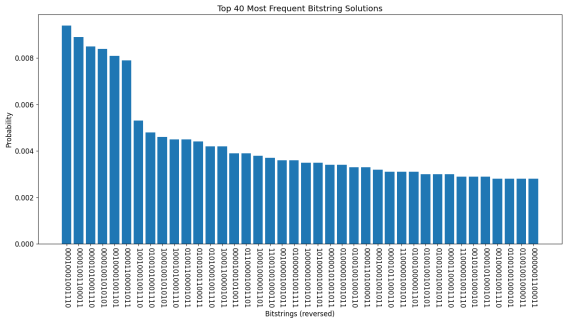
Figure 2: Visualization of sample VRP instance on 2D plane.

amplifying separation between distant nodes, improving numerical scaling for QUBO encoding. The resulting complete graph serves as the input to the QUBO formulation, which encodes routing costs, flow conservation, and depot constraints.

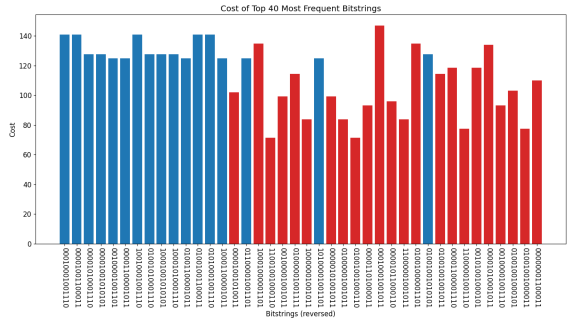
1.1 Feasibility of Solutions

A primary concern for constrained optimization is the fraction of outputs that represent valid routes (i.e., obey one-hot assignment of customers to vehicles, proper depot returns, etc.). Across noiseless simulations, approaches that either preserve feasibility by design or guide parameters along a structured schedule outperform vanilla QAOA. Qualitatively, Figures 3b and 3a show that vanilla QAOA (with random initial angles) yields a broad distribution of sampled bitstrings, but many high-probability solutions incur constraint violations (red-colored cost bars in Fig. 3a) indicating invalid routes. In contrast, with the linear ramp initialization (Fig. 3c, 3d), the algorithm concentrates more weight on feasible solutions (fewer red bars), demonstrating that a well-chosen initial parameter schedule steers the search towards the valid subspace.

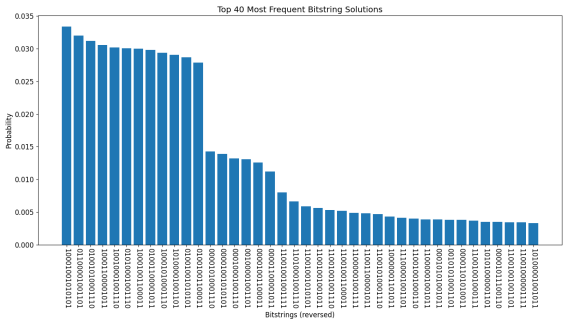
Similarly, methods explicitly designed to enforce feasibility show clear advantages in the simulations. Grover-Mixer QAOA starts in a Dicke state [19] and uses a feasible subspace mixer, so ideally all sampled bitstrings should have the same Hamming weight. In practice, we see from Table 1 that the GM-QAOA simulations indeed yielded a high proportion of valid solutions (e.g. 21% feasible at depths $p = 8$, versus $\approx 2\%$ for standard QAOA). Figure 3f confirms that the GM-QAOA's most probable solutions are all within the feasible subspace (no red bars in the top cost values), whereas the baseline QAOA at similar depth showed many invalid top samples. The Two-Step QAOA approach also improved feasibility relative to the single-stage



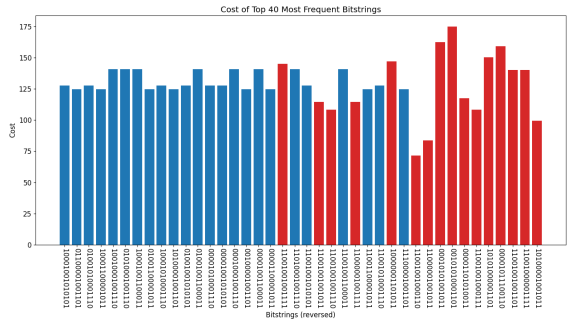
(a) Histogram of top 40 most frequently sampled bitstrings from QAOA simulation.



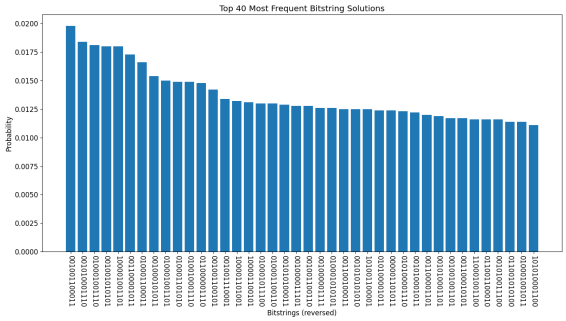
(b) Costs of top 40 most frequent bitstrings from QAOA simulation.



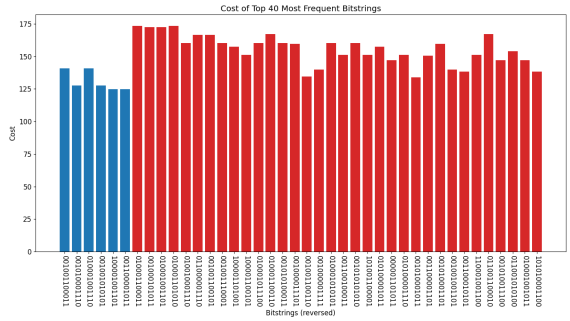
(c) Histogram of top 40 most frequently sampled bitstrings from LRI-QAOA simulation.



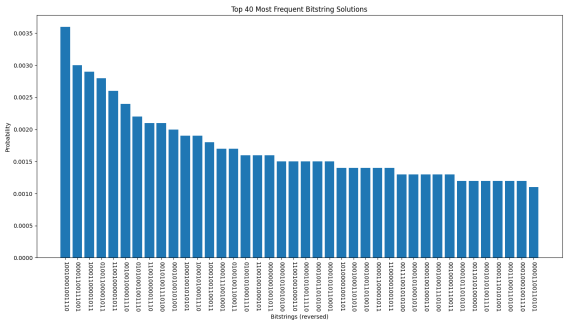
(d) Costs of top 40 most frequent bitstrings from LRI-QAOA simulation.



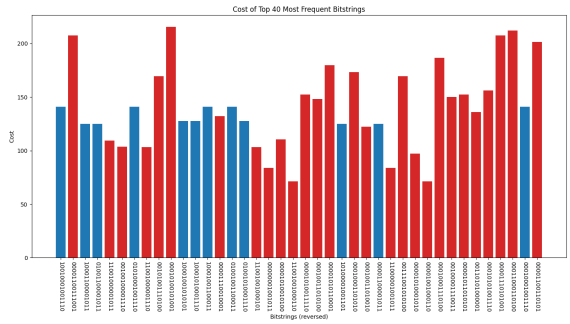
(e) Histogram of top 40 most frequently sampled bitstrings from GM-QAOA simulation.



(f) Costs of top 40 most frequent bitstrings from GM-QAOA simulation.



(g) Histogram of top 40 most frequent bitstrings from Two-Step QAOA simulation.



(h) Costs of top 40 most frequent bitstrings from Two-Step QAOA simulation.

Figure 3: Statevector simulations. (a),(b) QAOA initialized with $\gamma = \pi$, $\beta = \pi/2$, and Linear ramp initialization, $p = 6$. (c),(d) GM-QAOA, $p = 4$ and Two-step QAOA, $p = 8$. Constraint violations are colored red.

QAOA, though to a lesser extent than GM or LRI. Two-Step QAOA is designed to output only valid states after the second stage; however, our results suggest that it did not always remain strictly in the feasible manifold. We suspect this is due to imperfections in the intermediate state preparation or slight leakage from the constraint subspace, e.g., if the first stage does not perfectly project onto feasible states, the second stage might still explore some invalid configurations. For instance, at $p = 4$, two-step QAOA had 5.6% of samples feasible versus 2.4% for standard QAOA – an improvement, but still leaving many violations. Figure 3g and 3h qualitatively illustrate that the two-step variant's output distribution includes fewer invalid solutions than vanilla QAOA, yet some constraint violations persist, indicating the challenge of perfectly preparing and maintaining a complex constrained state on a small quantum circuit.

When moving to realistic sampling (QASM simulation) and especially to hardware, the feasibility ratio drops for all methods, as expected. On the Aer simulator (with 10k shots), sampling noise alone caused a significant decrease in measured feasibility – e.g., LRI-QAOA's feasible fraction fell from 47.6% (statevector) to 1.7% on Aer at $p = 8$, likely because sampling variability blurs the sharp distinctions in the loss landscape, reducing the optimizer's ability to resolve near-feasible states. The Grover mixer and two-step also saw their feasibility rates decrease significantly under shot noise (Table 1). Still, in the shot-based simulations we observed the same rank-order of performance: the linear-ramp and Grover-mixer variants consistently output more valid solutions than standard QAOA or two-step.

Owing to its low complexity, the LC-QAOA showed no significant advantage in low depth simulations. This is to be expected, as the aim of this method is to enable higher p QAOA on hardware. Nevertheless, it was able to provide the correct solution at rank 8 with $p = 6$ on the Aer simulation. Figure 4 summarizes the feasibility percentages for each variant on the QASM simulator across depth p, showing that in ideal conditions the regular QAOA performs well, and increasing p may not always yield the best solutions. The two-step method does benefit from increasing layers, likely due to much better feasible state preparation with higher depth. Likewise, GM-QAOA with Dicke state and short depth Dicke state initializations [20] (labeled SDGM-QAOA) show a higher performance at higher p.

1.2 Circuit Depth and Gate Complexity

One of the clearest distinctions among the QAOA variants is the complexity of the quantum circuit required to implement each ansätz, especially as measured by two-qubit gate depth (since multi-qubit gates like CNOT or CZ are the primary source of errors on superconducting hardware [21]). Our results show a stark contrast in circuit depths, with the Linear-Chain QAOA providing a significant advantage. Figure 5a plots the transpiled two-qubit gate depth for each method's circuit (at a fixed problem size) on the IBM_Fez device. The Linear-Chain QAOA had the shallowest circuit

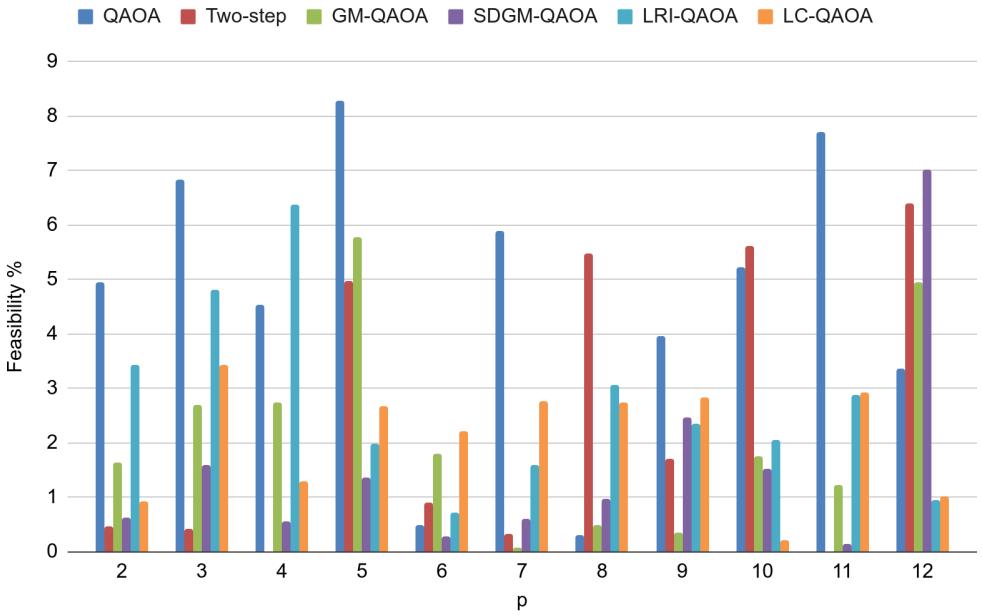
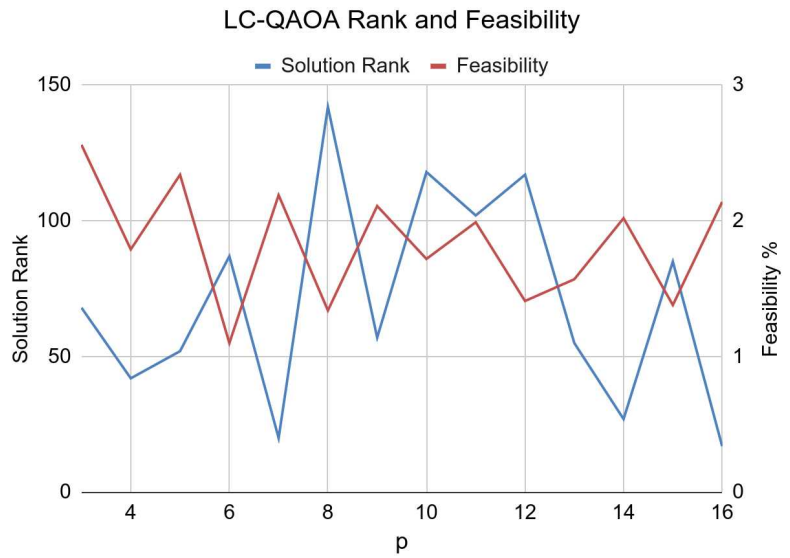


Figure 4: Feasibility percentage for QAOA variants on Aer simulator. SDGM-QAOA refers to GM-QAOA initialized with short depth Dicke state preparation from [20].

by far – by entangling only adjacent qubits along a chain, it avoids the need for long-range swaps and heavy gate overlap. By contrast, the Grover-mixer QAOA and Two-Step QAOA circuits were substantially deeper. Grover-mixer QAOA incurs extra depth from preparing the initial Dicke state and implementing global XY mixers that entangle distant qubits (which, on hardware, must be realized through sequences of nearest-neighbor swap gates). Two-Step QAOA doubles the number of QAOA layers (since it runs a constraint stage and then a cost stage), and while each stage individually might be simpler, the overall circuit had roughly two times the gates of a single-stage QAOA. Even with optimizations, we found that two-step circuits did not cancel out enough gates to overcome this doubling. Meanwhile, the GM-QAOA circuits are not plotted because even at $p = 2$ they have a two-qubit gate depth of over 3000, and they remained among the largest circuits in terms of gate count. In fact, the transpiler fails to execute for GM- and SDGM-QAOA for depth $p > 5$.

The standard QAOA and LRI-QAOA circuits fall in between these extremes. A vanilla QAOA ansätz with full connectivity mixers generates many non-neighbor interactions that the transpiler must decompose with swap gates, contributing to deep circuits on devices like IBM_Rensselaer's heavy-hex lattice. The linear-ramp initialized version uses the same circuit structure as standard QAOA, so its depth is essentially the same as the baseline QAOA for a given p. However, one benefit of LRI-QAOA is that it can sometimes achieve similar solution quality with a lower depth than random-initialized QAOA (since the initial ramp parameters are near optimal from the start). This means we might not need as large a p to reach a good

(a) Two-qubit gate depth for QAOA variants transpiled for IBM_Fez



(b) Optimum solution ranking and feasibility percentage for LC-QAOA with linear-ramp initialization and $XpXm$ dynamical decoupling on IBM_Rensselaer.

Figure 5: LC-QAOA transpiles to very shallow circuits while maintaining stable feasibility and favorable optimum solution rank against increasing p.

solution, indirectly reducing the circuit depth required. In our hardware experiments, we saw that a ramp-initialized QAOA at $p = 4$ could match or outperform a randomly initialized QAOA at $p = 6$ in terms of solution quality, effectively saving two layers

of gates.

Overall, Linear-Chain QAOA emerged as the most hardware-efficient circuit. As noted, it achieved the smallest two-qubit depth after compilation on both IBM devices. This shallowness directly contributes to its higher success on real hardware – fewer two-qubit gates mean fewer opportunities for errors to accumulate. In contrast, the Grover and Two-Step variants, while algorithmically intriguing, produced deep circuits that exceeded the error-tolerance of the hardware, explaining why their performance was severely limited by noise. Indeed, our experimentation highlights that the linear-chain ansätz offers a favorable trade-off: slightly reduced expressiveness (it cannot directly couple arbitrary qubits) in exchange for dramatically improved fidelity. The results support the notion that reducing circuit depth is critical for near-term quantum optimization. Even if an ansätz is theoretically powerful, it must be implementable with low error on real devices to actually outperform simpler methods. The linear-chain QAOA strikes this balance effectively by tailoring the ansätz to hardware constraints (topology and error rates), whereas two-step and Grover-mixer approaches, in their current form, struggle due to their circuit complexity.

1.3 Hardware behavior and error mitigation

Hardware execution reveals the decisive role of depth and error suppression. Without mitigation, feasibility on Rensselaer (Eagle) and Fez (Heron) is well below 1% for all methods at shallow (i.e., practically implementable) depths, reflecting the dominance of decoherence and two-qubit gate errors in the raw output distribution. This result is congruent with previous observations [13]. Nevertheless, with lightweight dynamical decoupling ($XpXm$) and parameter transfer from simulation, LRI-QAOA not only raises feasibility but also elevates the optimal route into the set of frequently sampled solutions. In this regime, LC-QAOA exhibits a consistent advantage because its linear-neighbor structure minimizes routing overhead and swap insertion during transpilation, achieving feasibility higher than 2% on Eagle hardware as shown in Fig. 5b. This pattern underscores a practical lesson for near-term optimization: inline error suppression that shortens the effective idle time and preserves fragile phases is more impactful than overhead-heavy error mitigation strategies.

The gap between simulation and hardware outcomes in our study underlines the importance of noise robustness. The limited connectivity and gate errors in quantum processors mean that even small QAOA circuits (12-14 qubits with a few layers) experience decoherence and crosstalk that scramble the algorithm's output distribution. As a result, the feasibility metrics that were high in simulation collapsed to nearly zero on the raw hardware runs. Our experiments with error suppression techniques provide insight into what helps and what hinders for QAOA type algorithms in the NISQ setting. We found that dynamical decoupling (DD) was particularly effective at improving QAOA results on hardware. By inserting periodic X-pulse sequences on idle qubits, DD protects qubits from accumulating phase errors while other oper-

ations occur. In our hardware sampling tests at depth $p = 4$, applying the $XpXm$ decoupling sequence boosted the fraction of feasible solutions and brought the optimal solution into reach. Hardware sampling was performed on regular QAOA with trained parameters obtained from Aer simulation. Table 2 quantifies this: on IBM_Fez, a circuit with XX decoupling yielded the optimal route at rank 16 with 1.14% feasible solutions, whereas with $XpXm$ decoupling the optimal route jumped to rank 6 and feasibility to 1.60%. A similar improvement was seen on IBM_Rensselaer (optimal rank improved from 235 to 15 with DD). This indicates that carefully chosen dynamical decoupling can substantially mitigate decoherence during the algorithm's runtime, effectively lengthening the coherence window to execute the QAOA circuit more faithfully.

In addition to decoupling, we experimented with Pauli twirling, readout error mitigation (TREX), and zero-noise extrapolation (ZNE), both individually and in combination (Table 3). The QAOA was optimized on hardware with parameters initialized from Aer simulation. Perhaps surprisingly, none of these advanced error mitigation strategies improved the results beyond what simple decoupling achieved. In fact, the data in Table 3 shows that the most aggressive mitigation (combining $XpXm$ + Pauli twirling + TREX + ZNE) resulted in the worst performance in terms of feasibility, which dropped to 0.60%, compared to 1.80% feasibility with only $XpXm$ DD. Intermediate levels of mitigation similarly underperformed: for example, adding Pauli twirling to $XpXm$ increased circuit randomness but yielded a lower feasible fraction (1.24%), and adding ZNE alone gave only a modest 1.34% feasibility. These results highlight a potential pitfall of piling on error mitigation: techniques like ZNE require running extended or modified circuits (to extrapolate the zero-noise value), which in a variational algorithm can distort the cost landscape or introduce additional sampling error, ultimately negating their benefit. Moreover, the fact that twirling and ZNE did not help in our QAOA runs suggests that stochastic noise mitigation may interfere with the variational optimization process. For instance, twirling randomizes certain error biases but also widens outcome distributions, making it harder for the classical optimizer to get a clear signal of improvement from the Estimator. In contrast, the dynamical decoupling is applied inline during the quantum circuit and does not require additional circuits or post-processing, so it does not hamper the iterative optimization loop. Our findings thus suggest that, for QAOA on current hardware, it is most productive to focus on error suppression at the circuit level (like DD and ansätz simplification) rather than heavy post-processing error mitigation. Indeed, the best hardware results we obtained (highest feasibility and correct solution appearance) came from using the simplest mitigation – $XpXm$ decoupling – on top of a hardware-efficient ansätz (LC-QAOA with linear ramp). This underscores that tackling noise at its source (by keeping circuits short and qubits idle as little as possible) is more effective than trying to "fix" noisy outputs after the fact.

1.4 Scaling to larger problems

To probe scaling under realistic constraints, we turn to a larger instance motivated by the two-qubit depth analysis in Fig. 6. We test LC-QAOA with linear-ramp initialization on a 5-node, 2-vehicle routing problem requiring 30 qubits (20 decision variables and 10 slack variables), a scale at which other QAOA variants cannot be executed reliably on hardware. The linear-chain ansätz remains compilable and benefits from increased depth: while small p shows weak performance, depths $p > 16$ reliably recover the optimal route on both quantum computers (Table 4). Despite low absolute feasibility, final sampling reveals the correct solution at $p = 19$ on IBM_Rensselaer and at $p = 18$ on IBM_Fez, with higher solution rank on Fez attributable to its lower two-qubit error rates and native RZZ implementation. These large instance experiments affirm that expressibility is not the limiting factor on current hardware; rather, hardware-aware circuit design and manageable depth govern success.

In fact, the results can further be improved by employing the Conditional Value at Risk (CVaR) cost function [17] during the optimization. CVaR is the expected value of the tail of the distribution of the distribution of costs sampled from the circuit. Unlike the extreme choice of using only the single lowest-cost sample (effectively $\alpha \approx 1/N_{\text{shots}}$), which yields a highly non-smooth, noisy landscape that impedes classical optimization, CVaR provides a smoother surrogate while still biasing the search toward high-quality solutions. The level α thus acts as a bias–variance knob: smaller α concentrates on the tail (greater exploitation) at the expense of smoothness, whereas larger α increases smoothness but reduces tail focus. In practice, α is selected to reflect expected hardware noise and sampling variance of the particular circuit.

The broader implication is that the co-design of algorithm, schedule, and compilation can unlock reliable performance at the edge of today's capabilities. Linear-ramp schedules shorten the path to high-quality parameters and can reduce the required number of layers to attain a target quality. LC-QAOA trims two-qubit depth by construction, improving mapping to heavy-hex topologies and significantly reducing swap overhead. On hardware tests, modest dynamical decoupling doubles effective feasibility and improves optimal-solution rank without complicating the variational loop. Combined with classical techniques like CVaR, these elements convert fragile proof-of-concept demonstrations into reproducible runs that recover the correct solution with non-negligible probability at unprecedented scales for this problem class. Taken together, these outcomes demonstrate a practical route for quantum optimization: prioritize hardware efficiency and structured parameterization, and minimize reliance on heavy post-processing.

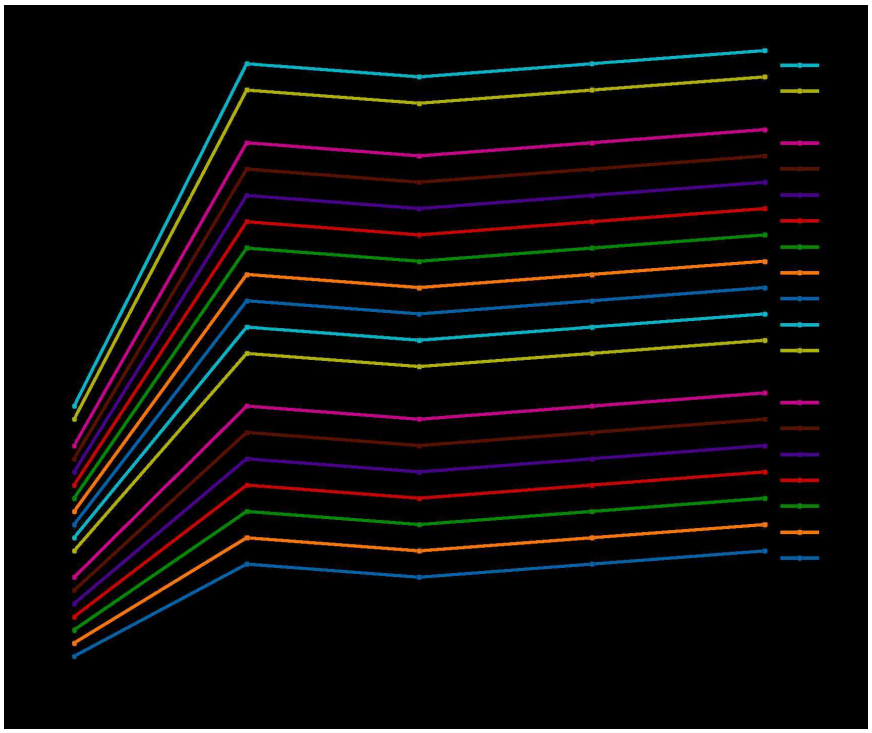


Figure 6: Transpiled circuit two-qubit depth of LC-QAOA as a function of problem size n, showing variation across depth parameters p. The sublinear growth is due to efficient transpilation of commuting RZZ gates.

2 Discussion

The results establish a pragmatic recipe for near-term routing on superconducting hardware: combine linear-ramp parameter schedules with a hardware-aligned linear-chain ansätz and prefer inline error suppression ($XpXm$) over heavy post-processing. In noiseless and shot-based simulations, feasibility-preserving or schedule guided variants concentrate probability on valid routes far more effectively than randomly initialized QAOA. On hardware, where raw feasibility is $< 1\%$ at shallow depths for all methods, the dominant differentiator is compiled two-qubit depth. LC-QAOA's neighbor-only couplings reduce swaps and cumulative error, yielding consistently higher valid-route fractions for the VRP and better optimal-solution ranks than more expressive but deeper alternatives.

The larger 30-qubit experiment clarifies that expressibility is not the binding constraint in this regime. Although small p underperforms, increasing depth beyond $p > 16$ enables reliable recovery of the optimal route on both backends, with higher rank on the lower-error device. This indicates that, given an ansätz that compiles efficiently and a schedule that steers parameters productively, additional layers translate into measurable gains even without elaborate error mitigation. Conversely, mitiga-

tion stacks that randomize or stretch circuits (twirling, TREX, ZNE) can counteract the variational feedback signal, lowering effective performance relative to simple decoupling.

These observations align with a co-design perspective: performance is governed jointly by algorithmic structure, compilation behavior, and device characteristics. Linear-chain truncation removes long-range ZZ terms that are costly to route on heavy-hex topologies. High level transpilation then exploits commuting structure to merge or parallelize gates, further shrinking depth. Linear-ramp schedules and CVaR objective reduce classical search overhead and appear transferable across depths and instances, making them suitable defaults for exploratory and scaling studies.

There are limitations. First, absolute feasibility on current hardware remains low, and recovery of the optimal route is probabilistic. Second, our instances are purposefully modest to permit repeated sampling and controlled comparisons. Extending to capacitated or time window VRPs will stress both width and depth. Third, results depend heavily on backend calibration and mapping. While LC-QAOA mitigates routing overhead, device-specific variability can shift optimal p and the benefits of decoupling. Practically, these findings suggest prioritizing circuit designs that keep two-qubit depth within the coherence budget, then investing in schedules and parameter transfer rather than heavy post-processing. For larger or more constrained logistics tasks, hybrid strategies are promising: use classical heuristics for decomposition or warm starts, apply LC-QAOA and linear ramp initialization on subproblems, and fuse solutions classically. Such workflows match current device capabilities while preserving a path to scale.

Future work should expand along three axes. *(i)* Problem realism: incorporate capacities, time windows, and multi-objective costs (e.g., emissions) while benchmarking encodings that balance qubit count against feasibility guarantees. *(ii)* Adaptive schedules and transfer: study schedule families that adapt to measured gradients on-device and quantify cross-instance transfer at fixed p. *(iii)* Hardware-aware compilation: co-optimize qubit placement, chain orientation, and dynamical decoupling timing. Comparative studies against VQE-style problem-inspired ansätze and annealing/CQM baselines on the same instances will help calibrate when QAOA's depth versus expressibility trade-off is most advantageous.

Overall, the takeaway for NISQ optimization is that simplicity which matches hardware beats theoretical expressiveness which hardware cannot faithfully realize. LC-QAOA with linear-ramp initialization, coupled with lightweight decoupling, provides a scalable and reproducible path to tackle routing instances at the edge of today's devices, and a foundation to build on as hardware and compilers improve.

3 Methods

The vehicle routing problem was formulated as a minimization problem using directed edges as decision variables. Constraints were converted to quadratic penalties and

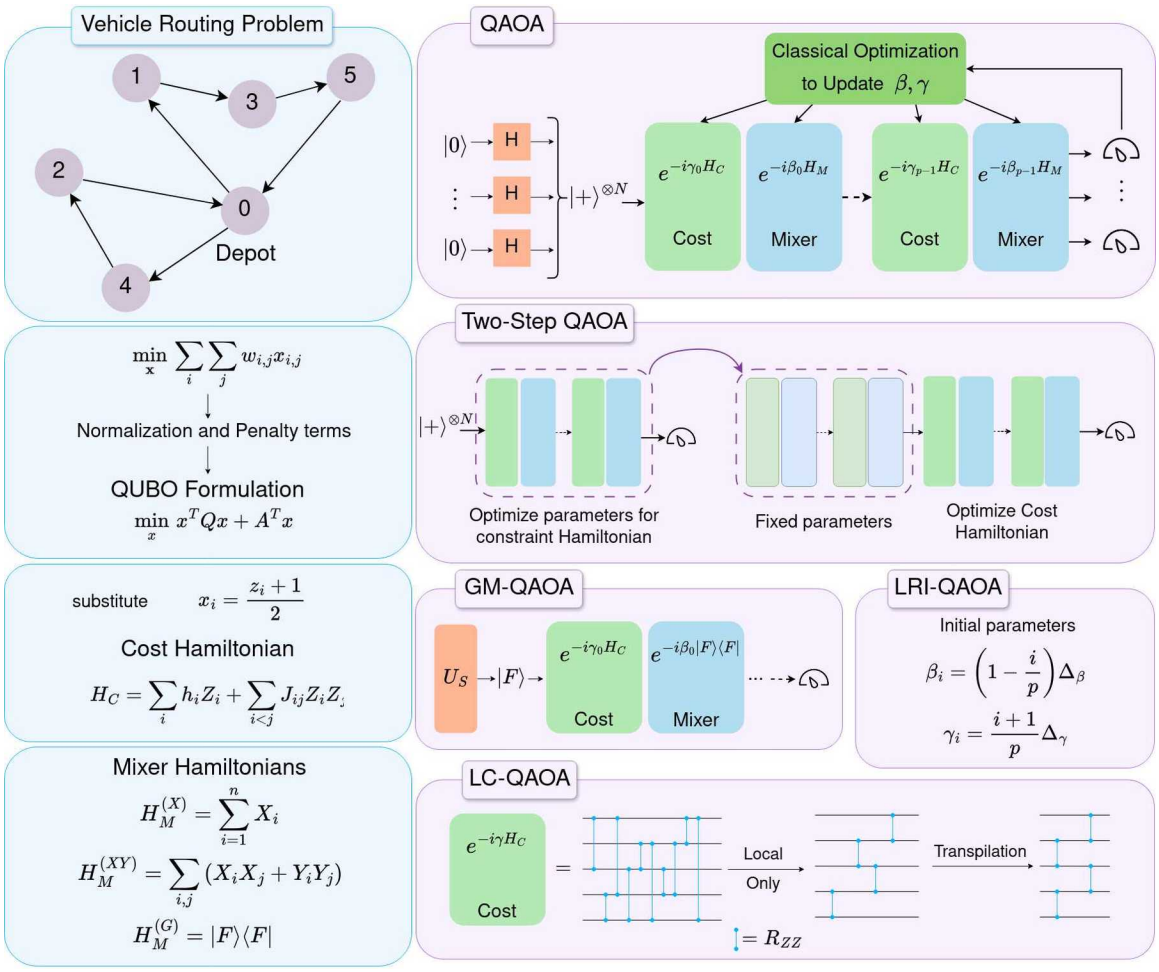


Figure 7: Overview of VRP mapping and QAOA variants compared in this study.

added to the cost function resulting in a QUBO. A change of binary to spin variables converts the QUBO into an Ising Hamiltonian. This representation was then used to create the circuits for the various QAOA methods studied in this paper. A broad summary of the process and a high level overview of the QAOA variants is shown in Figure 7.

3.1 Problem Formulation and Qubit Encoding

3.1.1 Vehicle Routing Problem

Following our previous work [13], we formulate the VRP as a binary decision variable minimization problem over the sum of edge costs. We assume that the vehicles start at the depot (node 0), customer demands are always met when the node is visited, and there are no time windows. Let $x_{i,j}$ be the binary variable indicating a directional link between nodes i and j, in the set of nodes $\mathcal{N}$, with the distance or cost in the

link as $w_{i,j}$. The objective function for the link based formulation is then:

$$\min_{\mathbf{x}} \sum_i \sum_j w_{i,j} x_{i,j} \quad i, j \in \mathcal{N}, \tag{1}$$

subject to the following constraints: that each customer is visited exactly once,

$$\sum_{j>0} x_{i,j} = 1 \quad \forall i > 0 \ ,$$

if a vehicle enters a node, it must leave,

$$\sum_j x_{i,j} = \sum_j x_{j,i} \quad \forall i \ ,$$

and there are exactly K vehicles leaving and entering the depot,

$$\sum_j x_{0,j} = K \ , \ \sum_i x_{i,0} = K \ .$$

In order to prevent disjoint loops that do not visit the depot, subtour elimination constraints must be imposed,

$$\sum_{i \in S} \sum_{j \notin S} x_{i,j} \geq 1, \quad \forall S \subset \mathcal{N}, \ 2 \leq |S| \leq n-1$$

where S is any subset of the node set $\mathcal{N}$, such that every S must have at least one connected link to some node in the complement set. Without these explicit constraints, subtours may appear in the solution which satisfy all the other constraints and could have lower cost than legitimate tours. For example, in an 8-node, 2-vehicle problem there could be the set of routes $0 \to 1 \to 2 \to 0$, $0 \to 3 \to 4 \to 0$, and a subtour $5 \to 6 \to 7 \to 5$. This configuration satisfies all constraints except subtour elimination.

3.1.2 Conversion to QUBO

We formulate the VRP as a 0-1 integer programming model in DOcplex, then convert it to a Qiskit [18] QuadraticProgram and finally to an Ising Hamiltonian used by QAOA. To run QAOA, we require a QUBO (quadratic unconstrained binary optimization). Using the quadratic program to QUBO function in the Qiskit optimization library, linear and quadratic penalty terms are added to the objective so the unconstrained minimizer respects feasibility.

Let the original cost be $f(x) = \sum_{i \neq j} w_{i,j} x_{i,j}$. Each equality constraint $g_k(x) = 0$ is folded into the objective as a quadratic penalty weighted with some constant λ:

$$\min_x f(x) + \lambda \sum_k \left(g_k(x)\right)^2 ,$$

with $x \in \{0,1\}^{n(n-1)}$. Because $x_i^2 = x_i$ for binary variables, each squared term expands to a quadratic polynomial, yielding a standard QUBO:

$$\min_x x^\top Q x + a^\top x + c.$$

As a rule of thumb, $\lambda > \sum_{i \neq j} |w_{i,j}|$ is considered sufficient as argued in [22], and in our previous work [13] we found that a factor of 2 resulted in better feasibility ratios for the optimization process. This heuristic ties λ to the problem's native cost scale so that violating a constraint is always more expensive than any beneficial reduction in travel cost. In practice, it should be large enough to dominate objective improvements but not so large that it causes numerical conditioning issues during transpilation and parameter optimization. Therefore, we set a global penalty

$$\lambda = 2 \sum_{i \neq j} |w_{ij}|$$

After conversion, we normalize the coefficients by dividing the terms with the magnitude of the largest term. This preserves argmin while improving numerical stability for optimization and hardware mapping.

The IBM CPLEX Python library was used to establish the classical baseline to certify the optimal solution for each VRP instance. CPLEX solves the mixed-integer model to proven optimality, and solve times are 1–2 seconds on a laptop-class CPU. We do not treat CPLEX wall-clock time as a head-to-head comparator with QAOA since our focus is feasibility and depth scaling on NISQ hardware.

3.1.3 Transformation to Ising Hamiltonian

The final QUBO is then converted to the Ising Hamiltonian by mapping binary variables to spins

$$x_i = \frac{1 - z_i}{2}, \quad z_i \in \{-1, +1\}.$$

Substituting into the normalized QUBO $x^\top \tilde{Q} x + \tilde{a}^\top x$ yields an Ising form

$$H(z) = \sum_i h_i z_i + \sum_{i<j} J_{ij} z_i z_j + \text{ const}$$

with coefficients

$$J_{ij} = \frac{\tilde{Q}_{ij}}{4}, \quad h_i = \frac{1}{2} \sum_{j \neq i} \tilde{Q}_{ij} - \frac{\tilde{a}_i}{2}$$

where the constant term can be discarded for the optimization process.

3.1.4 Separating Cost and Constraint Hamiltonians

For the Two-step QAOA variant, the Vehicle Routing Problem (VRP) QUBO is decomposed into separate cost and constraint components. Starting from the penalized formulation

$$\min_{x \in \{0,1\}^n} f(x) = C(x) + \lambda P(x),$$

where $C(x)$ represents the routing cost and $P(x)$ the aggregated quadratic penalties enforcing feasibility, we isolate these terms to obtain independent Hamiltonians: H_C (cost) and H_P (constraints). This separation enables distinct quantum evolutions corresponding to objective optimization and constraint enforcement.

In the two-step QAOA, the constraint Hamiltonian H_P is first optimized in a dedicated QAOA stage to identify parameters that effectively project the equal-superposition quantum state onto the feasible subspace. These optimized parameters are then frozen, and a second QAOA is performed using the cost Hamiltonian H_C, appended to the circuit to refine solutions within the feasible region.

3.1.5 Dicke State Preparation

Dicke states are a family of symmetric, entangled quantum states of multiple qubits important in quantum information, metrology, and quantum networks due to their entanglement properties and symmetry. These states are symmetric under qubit permutation and have a constant Hamming weight (number of 1s). Formally, an n-qubit Dicke state with k excitations (i.e., k qubits in state $|1\rangle$ and $n-k$ in $|0\rangle$) is defined as:

$$|D_k^n\rangle = \frac{1}{\sqrt{\binom{n}{k}}} \sum_{\text{perms}} |\underbrace{1\cdots1}_{k}\underbrace{0\cdots0}_{n-k}\rangle$$

where the sum is over all distinct permutations of k ones and $(n-k)$ zeros.

We employ an efficient method to generate Dicke states developed in [19]. While the same authors have proposed shorter depth circuits [20], the methods require hand-tuned non-scalable implementations of topology dependent recursive structures.

In the binary encoding, each qubit indicates whether a specific directed edge $i \to j$ is selected. The feasible subspace must therefore have a fixed Hamming weight, reflecting the total number of active edges. Specifically, since n nodes must be served and each of the K vehicles contributes an additional departure and arrival edge at the depot, the total number of ones in a valid configuration is $w = n + K - 1$. When combined with the Grover mixer, which preserves Hamming weight, amplitude is redistributed only among solutions of feasible Hamming weight. However, while a feasible bitstring must have this Hamming weight, any string of this Hamming weight is not guaranteed to be feasible.

3.1.6 Removal of nonadjacent ZZ interactions

When targeting NISQ hardware, non-local ZZ couplings inflate depth due to SWAP gate insertion. The linear-chain filter discards Pauli strings whose ZZ act on non-adjacent logical indices, retaining only $\{Z_i, Z_iZ_{i+1}\}$. In effect, the filter simply removes those terms from the list of Pauli operations. This reduces two-qubit depth substantially and aligns with linear-chain QAOA, at the cost of reduced expressibility.

4 QAOA Variants and Parameter Strategies

4.1 QAOA

The Quantum Approximate Optimization Algorithm (QAOA) is a hybrid quantum-classical algorithm designed for combinatorial optimization problems by leveraging quantum computing properties on a near-term quantum device in the NISQ era [1]. It stems from the intuition of the adiabatic theorem, which establishes that if the change in the Hamiltonian is gradual, the system remains in the ground state [23]. Given a driving Hamiltonian, H_D, with an easily prepared ground state and a problem Hamiltonian, H_P, the Hamiltonian representing the whole system is

$$H(s) = (1-s)H_D + sH_P$$

where s is increased in small steps from 0 to 1. In practice this is implemented as time evolution under H_D for a duration $(1 - j\Delta t)$ and H_P for duration $j\Delta t$, with j being incremented in every subsequent layer.

Instead of continuously evolving the Hamiltonian as in the adiabatic model, QAOA performs a finite number of parameterized steps, effectively approximating the gradual evolution with a layered, variational circuit. At each layer p, the system alternates between evolution under the problem (or cost) Hamiltonian H_P, which encodes the objective function to be minimized, and the driver (or mixer) Hamiltonian H_D, which promotes exploration of the state space. The overall variational ansätz after p layers is expressed as

$$|\psi_p(\boldsymbol{\gamma}, \boldsymbol{\beta})\rangle = \prod_{l=1}^{p} e^{-i\beta_l H_D} e^{-i\gamma_l H_P} |\psi_0\rangle ,$$

where $\boldsymbol{\gamma} = (\gamma_1, \ldots, \gamma_p)$ and $\boldsymbol{\beta} = (\beta_1, \ldots, \beta_p)$ are tunable parameters optimized by a classical routine to minimize the expected cost $\langle\psi_p| H_P |\psi_p\rangle$.

Through this hybrid process, QAOA inherits the physical grounding of adiabatic evolution while remaining compatible with noisy intermediate-scale quantum (NISQ) hardware. The alternation between cost and mixer unitaries provides a controllable balance between exploration (via H_D) and exploitation (via H_P), forming the foundation for subsequent variants such as two-step, Grover-mixer, and linear-chain QAOA, which adapt this framework for constrained and hardware-efficient optimization.

4.2 Two-Step QAOA

A significant challenge in applying the QAOA to combinatorial optimization problems is the formulation of constraints within the QUBO model. Typically, constraints, such as k-hot constraints where k out of n qubits must be in a specific state, are added to the cost function as penalty terms. This approach introduces additional tuning parameters that are difficult to balance, often leading to increased computational load and suboptimal solutions. If penalties are too small, infeasible states dominate the solution space, whereas excessively large penalties distort the cost landscape and hinder convergence. One approach is to use the Quantum Alternating Operator Ansatz, which allows for different mixer Hamiltonians, such as the XY mixer. The XY mixer can enforce these k-hot constraints as hard constraints, removing them from the cost function entirely.

Two-Step QAOA [24] can be used to prepare a state that encodes the required constraints by optimizing a QAOA circuit. This way constraint satisfaction can be done independently from objective minimization.

1. Constraint Satisfaction: The first QAOA is run using a standard X mixer and an easily prepared superposition state. The cost Hamiltonian for this step is simply the constraint equation itself, e.g., $H_{C_1} = (\sum_i x_i - k)^2$, without requiring penalty factors. This initial run generates an intermediate quantum state that satisfies the constraint conditions, effectively serving as an approximation of the required Dicke state.

2. Problem Optimization: The resulting constraint-satisfying state is then used as the initial state for a second QAOA, in which the cost Hamiltonian encodes only the objective function (e.g., total route cost in VRP) and the XY mixer preserves the Hamming weight of the generated feasible subspace.

This two-phase structure isolates constraint enforcement from cost minimization, improving convergence stability and scalability while reducing sensitivity to penalty tuning.

4.3 GM-QAOA

The core principle of GM-QAOA is to shift the complexity of the algorithm from the design of the mixing Hamiltonian to the initial state preparation [25]. The approach begins with a state preparation unitary, U_S, which creates an equal superposition of all feasible solutions, a state denoted as $|F\rangle$. For problems with a fixed Hamming-weight constraint, such as Max k-Vertex Cover, this initial state may be a Dicke state, $|D_k^n\rangle$, which can be prepared with $O(kn)$ depth. Instead of a traditional mixer, GM-QAOA uses a Grover-like selective phase shift operator as the mixer which can be implemented exactly by leveraging the state preparation unitary:

$$U_M(\beta) = e^{-i\beta|F\rangle\langle F|} = U_S(I - (1 - e^{-i\beta})|0\rangle\langle 0|)U_S^\dagger$$

where I is the identity matrix.

Similar to Two-step QAOA, the cost function now does not need to include the soft constraints because the mixer ensures that only feasible solutions are explored during optimization. An efficient preparation of the initial state is crucial for this method since that sub-circuit will be repeated multiple times in the overall GM-QAOA circuit.

However, generating a feasible subspace of states (i.e., satisfying all constraints) in the first place is akin to optimization, and is therefore nontrivial. Instead, we use Dicke states of predetermined Hamming weight to produce states with much higher probability of being feasible. The Grover mixer then preserves Hamming weight, and amplitude is redistributed only among these states. The cost Hamiltonian H_C acts as an oracle that selectively inverts the phase of states proportional to their route cost. Iterative applications of this oracle-mixer sequence amplify the amplitudes of low-cost feasible solutions while suppressing infeasible ones.

4.4 Linear Ramp Initialization

The study of QAOA parameters aimed at reducing classical overhead has revealed that parameter sets yielding strong performance on small problem instances tend to generalize to larger instances of the same class [26]. This behavior suggests a degree of instance independence in the QAOA loss landscape, thereby reducing the need for extensive parameter re-optimization when scaling to new problem sizes or higher depths. The performance of continuous parameter schedules has been studied over different parameter sizes and circuit depths to reveal similarity across many problem domains [27]. In fact, parameters found to optimize problems like bin packing have been successfully transferred to other combinatorial optimization problems like the traveling salesman problem [28]. The idea of fixed parameter schedules has been extensively tested numerically and experimentally for the linear ramp QAOA (LR-QAOA), showing that the protocol is robust to noise across many problems and problem sizes without fine-tuning [4]. In this scheme, the QAOA parameters are linearly interpolated across layers according to

$$\beta_i = \left(1 - \frac{i}{p}\right)\Delta_\beta, \quad \gamma_i = \frac{i+1}{p}\Delta_\gamma$$

where p denotes the circuit depth, and Δ_β and Δ_γ control the overall rotation scales of the driver and cost Hamiltonians, respectively.

In this study, we integrate the concepts of linear-ramp scheduling and parameter transfer to initialize QAOA circuits for evaluation on both simulators and real quantum hardware. The resulting Linear-Ramp Initialized QAOA (LRI-QAOA) demonstrates consistently improved performance in simulation, achieving higher feasibility ratios across all tested instances (Table 1).

However, the same initialization strategy was less effective on hardware, likely due to device noise and calibration variability. To address this, the final optimized parameters obtained from the Aer simulator runs were transferred to hardware experiments, where only the sampling step was executed. The transferred parameters immediately produced feasible solutions among the top 20 measured bitstrings on hardware, and further improved with hardware error suppression. Subsequent fine-tuning using these "warm-start" parameters further improved solution quality, raising optimal solutions to the top 10 samples in some trials and increasing the overall feasibility ratio.

4.5 Linear Chain QAOA

The depth of the QAOA circuit increases sharply with problem size, and importantly, the quadratic terms result in entangling gates between arbitrary qubits. On quantum hardware these two-qubit gates represent the greatest source of noise which is compounded when the qubits in question are not physically connected. For non-adjacent qubits, the entangling gates are implemented by a series of swap gates which bring the qubits together. In an effort to avoid this, linear chain QAOA (LC-QAOA) has been recently proposed [5], which entangles only neighboring qubits. This essentially means that Pauli strings are filtered to remove those where two Z terms are not adjacent. Restricting the cost Hamiltonian to a linear chain alters the summation structure of H_C and the effective connectivity of the ansätz.

The effect of this linear-chain filtering is to drastically reduce two-qubit gate depth at the expense of ignoring some problem correlations. The resulting circuit uses only nearest neighbor entangling gates (which, on hardware, can often be executed directly or with minimal routing). By construction, the transpiled circuit for a linear chain has far fewer long range interactions, meaning fewer SWAP insertions and a shorter overall depth. The downside is reduced expressiveness: the ansätz can no longer directly correlate qubits that were non-adjacent, potentially making it harder to represent the exact optimal solution if those couplings were important. However, the hope (borne out by recent work [5]) is that for certain problems, a linear topology still captures enough structure to find good solutions, while being much more robust on real devices. LC-QAOA is essentially a hardware-aware ansätz that trades off some theoretical optimality for a big gain in practical implementability.

Author Contributions

T.A. and R.K. jointly conceived the high-level research idea and T.A. conceived the detailed study design. T.A. developed the algorithms, implemented the algorithms, performed the numerical experiments, and analyzed the results. R.K. supervised the research, discussed the methodological development, and contributed to the interpretation of outcomes. T.A. drafted the manuscript, and R.K. provided revisions. Both authors reviewed and approved the final manuscript.

Funding

This work is funded through the IBM-RPI Future of Computing Research Collaboration. The funder also provided access to the premium usage of the IBM_Fez and IBM_Rensselaer quantum computer but played no role in study design, data collection, analysis and interpretation of data, or the writing of this manuscript. Both authors reviewed the final manuscript.

Competing Interests

All authors declare no financial or non-financial competing interests.

Data Availability

The data generated and used in this study is available at https://github.com/TalhaAzfar/LC_LRI_QAOA

Code Availability

The code to recreate the methods in this study is available at https://github.com/TalhaAzfar/LC_LRI_QAOA

References

[1] E. Farhi, J. Goldstone, S. Gutmann, A quantum approximate optimization algorithm, arXiv preprint arXiv:1411.4028 (2014).

[2] S. Hadfield, Z. Wang, B. O'gorman, E. G. Rieffel, D. Venturelli, R. Biswas, From the quantum approximate optimization algorithm to a quantum alternating operator ansatz, Algorithms 12 (2) (2019) 34.

[3] J. Cook, S. Eidenbenz, A. Bärtschi, The quantum alternating operator ansatz on maximum k-vertex cover, in: 2020 IEEE International Conference on Quantum Computing and Engineering (QCE), IEEE, 2020, pp. 83–92.

[4] J. Montanez-Barrera, K. Michielsen, Toward a linear-ramp qaoa protocol: evidence of a scaling advantage in solving some combinatorial optimization problems, npj Quantum Information 11 (1) (2025) 131.

[5] Z. Wang, J. Mandell, Y. Xu, J. Shi, A depth-independent linear chain ansatz for large-scale quantum approximate optimization, arXiv preprint arXiv:2509.17296 (2025).

[6] F. B. Maciejewski, B. G. Bach, M. Dupont, P. A. Lott, B. Sundar, D. E. B. Neira, I. Safro, D. Venturelli, A multilevel approach for solving large-scale qubo problems with noisy hybrid quantum approximate optimization, in: 2024 IEEE High Performance Extreme Computing Conference (HPEC), IEEE, 2024, pp. 1–10.

[7] L. Zhu, H. L. Tang, G. S. Barron, F. Calderon-Vargas, N. J. Mayhall, E. Barnes, S. E. Economou, Adaptive quantum approximate optimization algorithm for solving combinatorial problems on a quantum computer, Physical Review Research 4 (3) (2022) 033029.

[8] F. B. Maciejewski, J. Biamonte, S. Hadfield, D. Venturelli, Improving quantum approximate optimization by noise-directed adaptive remapping, arXiv preprint arXiv:2404.01412 (2024).

[9] G. Clarke, J. W. Wright, Scheduling of vehicles from a central depot to a number of delivery points, Operations research 12 (4) (1964) 568–581.

[10] L. Palackal, B. Poggel, M. Wulff, H. Ehm, J. M. Lorenz, C. B. Mendl, Quantum-assisted solution paths for the capacitated vehicle routing problem, in: 2023 IEEE International Conference on Quantum Computing and Engineering (QCE), Vol. 1, IEEE, 2023, pp. 648–658.

[11] N. Xie, X. Lee, D. Cai, Y. Saito, N. Asai, H. C. Lau, A feasibility-preserved quantum approximate solver for the capacitated vehicle routing problem, Quantum Information Processing 23 (8) (2024) 291.

[12] A. Lucas, Ising formulations of many np problems, Frontiers in physics 2 (2014) 5.

[13] T. Azfar, O. M. Raisuddin, R. Ke, J. Holguin-Veras, Quantum-assisted vehicle routing: Realizing qaoa-based approach on gate-based quantum computer, arXiv preprint arXiv:2505.01614 (2025).

[14] M. Borowski, P. Gora, K. Karnas, M. Błajda, K. Król, A. Matyjasek, D. Burczyk, M. Szewczyk, M. Kutwin, New hybrid quantum annealing algorithms for solving vehicle routing problem, in: International Conference on Computational Science, Springer, 2020, pp. 546–561.

[15] S. Feld, C. Roch, T. Gabor, C. Seidel, F. Neukart, I. Galter, W. Mauerer, C. Linnhoff-Popien, A hybrid solution method for the capacitated vehicle routing problem using a quantum annealer, Frontiers in ICT 6 (2019) 13.

[16] J. B. Holliday, D. Blount, E. Osaba, K. Luu, Advanced quantum annealing approach to vehicle routing problems with time windows, arXiv preprint arXiv:2503.24285 (2025).

[17] P. K. Barkoutsos, G. Nannicini, A. Robert, I. Tavernelli, S. Woerner, Improving variational quantum optimization using cvar, Quantum 4 (2020) 256.

[18] A. Javadi-Abhari, M. Treinish, K. Krsulich, C. J. Wood, J. Lishman, J. Gacon, S. Martiel, P. D. Nation, L. S. Bishop, A. W. Cross, B. R. Johnson, J. M. Gambetta, Quantum computing with Qiskit (2024). arXiv:2405.08810, doi:10.48550/arXiv.2405.08810.

[19] A. Bärtschi, S. Eidenbenz, Deterministic preparation of dicke states, in: International Symposium on Fundamentals of Computation Theory, Springer, 2019, pp. 126–139.

[20] A. Bärtschi, S. Eidenbenz, Short-depth circuits for dicke state preparation, in: 2022 IEEE International Conference on Quantum Computing and Engineering (QCE), IEEE, 2022, pp. 87–96.

[21] M. Kjaergaard, M. E. Schwartz, J. Braumüller, P. Krantz, J. I.-J. Wang, S. Gustavsson, W. D. Oliver, Superconducting qubits: Current state of play, Annual Review of Condensed Matter Physics 11 (1) (2020) 369–395.

[22] S. Harwood, C. Gambella, D. Trenev, A. Simonetto, D. Bernal, D. Greenberg, Formulating and solving routing problems on quantum computers, IEEE Transactions on Quantum Engineering 2 (2021) 1–17.

[23] E. Farhi, J. Goldstone, S. Gutmann, M. Sipser, Quantum computation by adiabatic evolution, arXiv preprint quant-ph/0001106 (2000).

[24] Y. Minato, Two-step qaoa: Enhancing quantum optimization by decomposing one-hot constraints in qubo formulations, arXiv preprint arXiv:2408.05383 (2024).

[25] A. Bärtschi, S. Eidenbenz, Grover mixers for qaoa: Shifting complexity from mixer design to state preparation, in: 2020 IEEE International Conference on Quantum Computing and Engineering (QCE), IEEE, 2020, pp. 72–82.

[26] F. G. Brandao, M. Broughton, E. Farhi, S. Gutmann, H. Neven, For fixed control parameters the quantum approximate optimization algorithm's objective function value concentrates for typical instances, arXiv preprint arXiv:1812.04170 (2018).

[27] V. Kremenetski, A. Apte, T. Hogg, S. Hadfield, N. M. Tubman, Quantum alternating operator ansatz (qaoa) beyond low depth with gradually changing unitaries, arXiv preprint arXiv:2305.04455 (2023).

[28] J. Montanez-Barrera, D. Willsch, K. Michielsen, Transfer learning of optimal qaoa parameters in combinatorial optimization, Quantum Information Processing 24 (5) (2025) 129.

List of Tables

1 Simulation comparisons 27

2 Effects of dynamical decoupling types on **sampling** from the QAOA circuit on quantum hardware with best known Aer learned parameters, $p = 4$. 28

3 Effects of error suppression and mitigation on **optimizing** the QAOA on quantum hardware initialized with best known Aer learned parameters, $p = 4$ on IBM_Fez. 29

4 LC-QAOA with linear-ramp initialization for the 5-node, 2-vehicle problem compared with and without CVaR. Dynamical decoupling $XpXm$ is applied for all runs. 30

Table 1: Comparative results across QAOA variants and simulation backends. Solution time is in wall-clock seconds, and Rank refers to the position of the correct bitstring after sorting for sample frequency.

$p = 8$		QAOA	Two-Step	GM-QAOA	LRI-QAOA	LC-QAOA
Statevector	% feasible	2.4	4.43	21.38	47.6	2.86
	Rank	7	3	4	1	19
	Time	51	160	634	79	55
	Iterations	528	647	862	1057	1217
Aer	% feasible	1.5	0.13	6.7	1.68	1.25
	Rank	759	*	1	20	104
	Time	193	159	184	207	170
	Iterations	136	258	106	138	172
$p = 6$		QAOA	Two-Step	GM-QAOA	LRI-QAOA	LC-QAOA
Statevector	% feasible	15.4	0.94	16.33	17.11	0.67
	Rank	5	5	1	5	85
	Time	26	122	693	29	15.4
	Iterations	260	247	1256	539	433
Aer	% feasible	3.76	0.29	1.79	3.2	2.34
	Rank	11	364	40	4	8
	Time	114	196	111	195	89
	Iterations	96	1302	80	154	122
$p = 4$		QAOA	Two-Step	GM-QAOA	LRI-QAOA	LC-QAOA
Statevector	% feasible	2.4	5.6	10.72	25.54	3.94
	Rank	1	2	1	2	28
	Time	28	101	157	15	10.6
	Iterations	260	262	453	326	423
Aer	% feasible	2.77	0.4	3.71	6.47	1.57
	Rank	18	976	3	2	428
	Time	68	59	58	88	34.5
	Iterations	59	146	60	98	58

* Solution not found in top 1000 bitstrings

Table 2: Effects of dynamical decoupling types on **sampling** from the QAOA circuit on quantum hardware with best known Aer learned parameters, $p = 4$.

Quantum computer	Decoupling sequence	Solution rank	Feasibility %
IBM_Fez	XX	16	1.14
	$XpXm$	6	1.60
	$XY4$	133	1.42
IBM_Rensselaer	XX	235	0.69
	$XpXm$	15	0.80
	$XY4$	1478	0.40

Table 3: Effects of error suppression and mitigation on **optimizing** the QAOA on quantum hardware initialized with best known Aer learned parameters, $p = 4$ on IBM_Fez.

Decoupling sequence	None	$XpXm$	$XpXm$	$XpXm$	$XpXm$	$XpXm$
Pauli Twirling	✘	✘	✓	✘	✓	✓
TREX	✘	✘	✘	✘	✓	✓
ZNE	✘	✘	✘	✓	✘	✓
Solution Rank	291	45	1376	57	483	27
Feasibility %	1.51	1.80	1.24	1.34	0.87	0.60

Table 4: LC-QAOA with linear-ramp initialization for the 5-node, 2-vehicle problem compared with and without CVaR. Dynamical decoupling $XpXm$ is applied for all runs.

	$p = 17$	$p = 18$	$p = 19$	$p = 20$
IBM_Rensselaer				
Rank	–	–	2759	2857
Feasibility %	0.2	0.3	0.2	0.5
Rank (CVaR)	–	422	191	4721
Feasibility % (CVaR)	3	4.95	0.4	2.97
IBM_Fez				
Rank	-	1169	-	4358
Feasibility %	0.2	0.5	0.37	0.6
Rank (CVaR)	–	–	1150	38
Feasibility % (CVaR)	1.98	4.72	2.94	2.88